

Documentación técnica del Chatbot del Día de Muertos

Proyecto universitario — documentación técnica

Dato	Valor
Proyecto	Chatbot del Día de Muertos en México
Alcance	Día de Muertos en México
Base de conocimiento	50 entradas, 738 palabras clave
Fuentes documentadas	29 fuentes institucionales
Fecha del documento	27/09/2026

1. Arquitectura general

El sistema tiene cuatro capas, y cada una vive en archivos distintos para poder explicarlas por separado. No hay servidor propio, ni base de datos, ni servicios externos.

- **Interfaz:** components/Chatbot.tsx, el panel de chat escrito en React.
- **Lógica:** lib/motor.ts y lib/buscador.ts, que decide qué respuesta corresponde.
- **Información:** data/conocimiento.json, con las entradas del chatbot.
- **Fuentes:** data/fuentes.json, con la documentación de las fuentes.

2. Tecnologías utilizadas

Next.js en su versión 16 con App Router, React 19, TypeScript y CSS Modules. Todo el código se ejecuta en el navegador salvo el módulo de búsqueda, que es TypeScript puro y se puede ejecutar también desde Node para las pruebas. No hay dependencias de pago, ni APIs externas, ni librerías de inteligencia artificial.

3. Estructura de carpetas

Ruta	Contenido
app/page.tsx	Página principal: solo monta el componente de chat
app/layout.tsx	Estructura HTML, idioma español y metadatos
app/globals.css	Estilos generales del documento
components/Chatbot.tsx	Interfaz del chat: mensajes, entrada y botón
components/Chatbot.module.css	Estilos de la interfaz (CSS Module)
lib/motor.ts	Motor de búsqueda: normalización, puntaje y umbral
lib/buscador.ts	Carga los datos y arma la respuesta final con sus fuentes

Ruta	Contenido
lib/tipos.ts	Definiciones de tipos de TypeScript
data/conocimiento.json	Base de conocimiento del chatbot
data/fuentes.json	Fuentes con institución, URL, uso y justificación
data/evaluacion.json	Preguntas de evaluación por nivel
scripts/evaluacion.mjs	Script que ejecuta la evaluación y genera el reporte
scripts/generar_pdfs.py	Script que genera los tres documentos PDF
docs/evaluacion.md	Reporte de resultados de la evaluación
docs/*.pdf	Los tres documentos entregables

4. Función de cada archivo importante

Archivo	Qué hace exactamente
lib/motor.ts	Contiene el algoritmo, sin leer ningún archivo de datos. Expone <code>normalizar()</code> , <code>pesosPorPalabraClave()</code> , <code>puntuarEntrada()</code> y <code>elegirEntrada()</code> . Gracias a esa separación, el mismo motor se usa en la aplicación y en el script de pruebas.
lib/buscador.ts	Importa los dos JSON, llama al motor y arma el texto final: añade la línea de fuentes, avisa si hay un empate entre temas y devuelve la respuesta.
components/Chatbot.tsx	Mantiene el estado de la conversación, envía la pregunta al buscador y muestra la respuesta con sus fuentes. Si la respuesta es de fuera de alcance, la muestra con otro estilo.
data/conocimiento.json	Las 50 entradas. Cada una tiene id, tema, categoría, preguntas de ejemplo, respuesta, palabras clave, fuentes y prioridad.

5. Funcionamiento de la interfaz

Al abrir la página, el componente muestra un mensaje inicial que explica de qué trata el chatbot y cuatro sugerencias que se pueden pulsar. Cada sugerencia es una pregunta escrita por el estudiante. Cuando el usuario escribe y envía, la pregunta se guarda en el estado, se consulta el buscador y se añade la respuesta. Al final de cada respuesta aparecen las fuentes, con su nombre y su enlace, para que se pueda comprobar el dato. El botón de limpiar conversación devuelve el chat al estado inicial.

6. Funcionamiento del procesamiento de preguntas

El buscador trabaja en cinco pasos, todos explicables:

- **Normalización.** Se pasa la pregunta a minúsculas, se le quitan los acentos y los signos de interrogación, y se reducen los espacios. Así, Día de Muertos, día de muertos y ¿DÍA DE MUERTOS? se comparan igual.
- **Filtro de dominio.** Si la pregunta no contiene ningún término del vocabulario del tema, se rechaza de inmediato. Ese filtro evita que una pregunta como cuál es la capital de Francia se interprete como una pregunta sobre una región de México.
- **Cálculo de pesos.** A cada palabra clave se le asigna un peso: las que aparecen en pocas entradas valen más, porque son más específicas, y las frases de varias palabras reciben un poco más.

- **Puntaje de cada entrada.** Se suma el peso de las palabras clave que aparecen en la pregunta. Se ordenan las entradas de mayor a menor puntaje.
- **Umbral.** Si la mejor entrada no llega al umbral, el chatbot responde que no tiene información suficiente. No hay intento de adivinar.

7. Estructura de la base de conocimiento

Una entrada de data/conocimiento.json tiene esta forma:

Campo	Qué significa
id	Identificador único del tema
tema	Nombre del tema, por ejemplo El cempasúchil
categoría	Grupo al que pertenece: definición, fechas, elementos, origen, comparativa, regional, mitología, actualidad, patrimonio o significado
preguntas_ejemplo	Preguntas típicas que deberían llegar a esta entrada
respuesta	Texto que se muestra al usuario, redactado para el profesor
palabras_clave	Términos y frases que usa el buscador para comparar
fuentes	Identificadores de las fuentes que sustentan la respuesta
prioridad	Criterio de desempate cuando dos entradas empatan

Las categorías que existen en la base son: actualidad, adultos, calaveras, comparativa, definición, elementos, fechas, influencia-europea, influencia-indígena, manualidades, mascotas, mitología, mitos, niños, ofrenda, origen, patrimonio, práctico, proyecto, regional, significado.

8. Sistema de búsqueda y coincidencia

El algoritmo es de coincidencia de palabras clave con puntaje ponderado. Se eligió este enfoque porque cumple las tres condiciones del proyecto: no necesita servicios externos, es determinista y se puede explicar en un minuto. No hay vectores, ni embeddings, ni modelos.

Fórmula del puntaje de una entrada:

puntaje = suma de los pesos de las palabras clave que aparecen en la pregunta

Y el peso de una palabra clave es:

peso = $1 + 1 / (\text{número de entradas que contienen esa palabra}) + 0,5$ por cada palabra adicional si la clave es una frase

Un ejemplo concreto ayuda a entenderlo. En la pregunta ¿Por qué se colocan veladoras?, la palabra veladoras aparece en una sola entrada, así que vale 2 puntos. La entrada Veladoras y velas gana con 2 puntos, supera el umbral y se muestra. En la pregunta cuál es la capital de Francia no hay ni una coincidencia útil, así que el chatbot rechaza.

Cuando dos entradas empatan, se decide con el campo prioridad, y si también empatan se elige la primera en orden alfabético, para que el resultado sea siempre el mismo.

9. Manejo de preguntas desconocidas

Hay tres formas de que una pregunta sea rechazada, y todas llevan a la misma respuesta:

- La pregunta no contiene ningún término del vocabulario del chatbot.
- La pregunta no tiene ninguna coincidencia con las palabras clave.
- La mejor entrada existe, pero su puntaje queda por debajo del umbral.

El mensaje que se muestra es fijo: No tengo información suficiente sobre ese tema dentro de mi base de conocimiento. Mi especialidad es el Día de Muertos en México. Esa decisión de diseño es lo que impide que el sistema invente información.

10. Integración de las fuentes

Las fuentes están en `data/fuentes.json`, con `institucion`, `documento`, `url`, `verificación`, `información obtenida`, `justificación` y a qué parte del chatbot da soporte. Cada entrada de conocimiento declara los identificadores de sus fuentes, y `lib/buscador.ts` los resuelve contra la lista para poder mostrar el nombre y el enlace debajo de cada respuesta. De ese modo, cualquier afirmación del chatbot se puede rastrear hasta una fuente institucional con nombre y URL.

El script de evaluación comprueba además que no haya referencias rotas, es decir, que ninguna entrada cite una fuente que no exista en el archivo.

11. Flujo completo de una pregunta

Paso	Dónde ocurre	Qué sucede
1	<code>components/Chatbot.tsx</code>	El usuario escribe y pulsa Enviar
2	<code>components/Chatbot.tsx</code>	Se guarda la pregunta en el estado y se llama a <code>responder()</code>
3	<code>lib/buscador.ts</code>	Se cargan los datos y se delega en el motor
4	<code>lib/motor.ts</code>	Se normaliza la pregunta y se aplica el filtro de dominio
5	<code>lib/motor.ts</code>	Se calculan pesos y se puntúa cada entrada
6	<code>lib/motor.ts</code>	Se ordena por puntaje y se aplica el umbral
7	<code>lib/buscador.ts</code>	Se arma el texto final y se resuelven las fuentes
8	<code>components/Chatbot.tsx</code>	Se pinta la respuesta con sus fuentes y enlaces

12. Cómo ejecutar el proyecto localmente

Se necesita Node.js 20 o superior. Los pasos son:

- `git clone` del repositorio, o simplemente abrir la carpeta del proyecto.
- `npm install` para instalar las dependencias.
- `npm run dev` para levantar el servidor de desarrollo.
- Abrir `http://localhost:3000` en el navegador.
- `npm run evaluar` para ejecutar las pruebas y regenerar el reporte.

- `npm run build` para compilar la versión de producción.

No hace falta configurar ninguna variable de entorno, ninguna llave de API ni ningún servicio externo.
Si el proyecto se acaba de descargar, `npm install` es el único paso que requiere conexión a internet.